

Loop closure và nhất quán toàn cục (Loop Closure and Global Consistency)

PHỤ THUỘC A/03-lie-group-va-toi-uu-tren-da-tap ($Sim(3)$ và log của sai lệch tư thế); A/04-uoc-luong-map-va-bundle-adjustment (M-estimator, GNC); A/05-observability-gauge-va-suy-bien (số bậc tự do của pose graph theo cấu hình cảm biến); B/07-dac-trung-va-data-association (perceptual aliasing — ở đây nó xuất hiện lại với hậu quả nghiêm trọng hơn nhiều); B/09-paradigm-uoc-luong-back-end (pose graph).

TÓM TẮT

Odometry trôi, và trôi đơn điệu — không cơ chế nào trong B/09 tự sửa được điều đó. Loop closure là cơ chế duy nhất: khi hệ thống nhận ra nó đã quay lại một chỗ từng đi qua, nó thêm một ràng buộc nổi hiện tại với quá khứ, và ràng buộc ấy phân phối lại sai số tích lũy trên toàn quỹ đạo. Đây là thứ biến *odometry* thành *SLAM*. Nội dung có bốn phần. Một: nhận ra chỗ cũ — từ túi từ trực quan tới các mô hình học được — và vì sao đây là một bài toán truy hồi chứ không phải một bài toán khớp. Hai: kiểm định hình học, tăng bắt buộc biến một ứng viên thành một ràng buộc, và vì sao ngưỡng của nó phải bảo thủ hơn mọi ngưỡng khác trong hệ thống. Ba: áp dụng ràng buộc — $Sim(3)$ cho mono, pose graph, và việc hoà nhập kết quả với luồng tracking đang chạy. Bốn, và là phần quan trọng nhất: back-end bền, tức làm sao sống sót khi một ràng buộc sai lọt qua. Nếu chỉ nhớ một điều: *đây là chương duy nhất trong cây mà một lỗi đơn lẻ phá huỷ toàn bộ sản phẩm của hàng giờ làm việc* — nên chi phí của hai loại lỗi bất đối xứng tới mức mọi thiết kế phải xoay quanh sự bất đối xứng đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **loop closure** — việc nhận ra rằng vị trí hiện tại trùng với một vị trí đã đi qua, và thêm một ràng buộc tương ứng vào bài toán ước lượng. Gồm hai bước tách rời: *phát hiện* (nhận ra chỗ cũ) và *đóng* (áp dụng ràng buộc).
- **trôi (drift)** — sai số tích lũy của odometry, tăng đơn điệu theo quãng đường đi. Không cơ chế odometry nào tự sửa được trôi, vì nó không có thông tin nào về vị trí tuyệt đối.
- **túi từ trực quan (bag of visual words)** — biểu diễn một ảnh bằng histogram tần suất của các “từ” — các cụm descriptor đã học trước. Cho phép so sánh hai ảnh bằng một vectơ thưa thay vì bằng hàng trăm descriptor.
- **TF-IDF** — trọng số cho mỗi từ: từ xuất hiện nhiều trong ảnh này (term frequency) và hiếm trong toàn bộ kho (inverse document frequency) thì mang nhiều thông tin phân biệt hơn.
- **perceptual aliasing** — hai địa điểm khác nhau trông giống nhau tới mức không phân biệt được bằng vẻ ngoài. Đây là nguyên nhân gốc của loop closure sai, và nó là quy luật chứ không phải ngoại lệ trong môi trường nhân tạo.
- **kiểm định hình học (geometric verification)** — bước kiểm một ứng viên loop closure bằng cách tìm một phép biến đổi nhất quán giữa hai khung. Đây là tầng phòng thủ chính chống lại perceptual aliasing.
- **back-end bền (robust back-end)** — họ kỹ thuật cho phép bài toán tối ưu sống sót khi một số cạnh là sai, bằng cách cho phép bộ giải “tắt” chúng thay vì buộc phải thoả mãn chúng.

CÂU HỎI MỞ ĐẦU

- Q1. Loop closure sai làm gập cả bản đồ; loop closure bỏ lỡ chỉ làm hệ thống trôi thêm một chút. Ước lượng tỉ số chi phí giữa hai loại lỗi, rồi nói nó quyết định gì về thiết kế.
- Q2. Kiểm định hình học đã cho một phép biến đổi với hai trăm inlier. Vì sao đó vẫn chưa đủ để chấp nhận, và nêu phép kiểm bổ sung.
- Q3. Hành lang kho hàng có ba mươi kệ giống hệt nhau, cách nhau ba mét. Mọi phương pháp dựa trên vẽ ngoài đều thất bại ở đây. Nêu hai nguồn thông tin khác vẫn phân biệt được, và nói vì sao chúng không phải lời giải hoàn chỉnh.
- Q4. SLAM mono cần $Sim(3)$ cho loop closure còn SLAM stereo chỉ cần $SE(3)$. Nếu bạn dùng $SE(3)$ cho mono thì chuyện gì xảy ra cụ thể — không phải “kém chính xác hơn” mà là gì?
- Q5. Back-end bền cho phép bộ giải tắt các cạnh nghi ngờ. Nghe như nó giải quyết hoàn toàn bài toán loop closure sai. Nêu chính xác điều kiện để nó hoạt động, và nói khi nào điều kiện đó bị vi phạm.

1.1 Đặt vấn đề

Mọi bộ ước lượng ở B/09 có chung một tính chất: sai số của chúng *tích lũy*. Mỗi bước thêm một chút sai số, và không có gì kéo nó về. Với một robot chạy tám tiếng trong một nhà kho, sai số cuối ngày có thể là hàng chục mét — đủ để bản đồ trở nên vô dụng.

Điều làm vấn đề này đặc biệt khó chịu: *robot thường xuyên quay lại chỗ cũ*. Nó đi qua cùng một lối đi mười lần một ngày. Ở lần thứ mười, nó “biết” rằng nó đang ở chỗ đã đi qua — theo nghĩa thông tin đó có trong dữ liệu — nhưng nếu không ai nói cho nó, nó sẽ tiếp tục vẽ một lối đi thứ mười cách lối đi thứ nhất năm mét.

Bài toán gốc: phát hiện việc quay lại, và dùng nó để sửa. Đây là chữ “M” trong SLAM — không có nó, ta chỉ có odometry.

Ai gặp bài toán này? Mọi hệ chạy lâu trong không gian hữu hạn. Với một xe tự lái đi một chiều trên đường cao tốc, loop closure ít quan trọng; với robot dịch vụ, nó là yêu cầu sống còn.

Trước khi có các phương pháp truy hồi hiệu quả, cách làm là so khung hiện tại với *mọi* khung quá khứ. Chi phí tuyến tính theo số khung mỗi lần, tức bậc hai theo thời gian — không khả thi. Tút từ trực quan giải quyết bằng cách biến mỗi ảnh thành một vectơ thưa và dùng chỉ mục ngược, đưa chi phí truy hồi xuống gần hằng số [1].

Nhưng vấn đề thật không phải chi phí. Đó là *perceptual aliasing*, và nó nghiêm trọng hơn ở đây so với ở B/07 vì hai lý do. Thứ nhất, không gian tìm kiếm lớn hơn nhiều — ta so với toàn bộ bản đồ, không chỉ với khung trước. Thứ hai, và quan trọng hơn, *hậu quả lớn hơn nhiều*. Một khớp điểm sai giữa hai khung liền kề tạo ra một phần dư lớn mà M-estimator hạ trọng số. Một loop closure sai tạo ra một ràng buộc nói “hai chỗ cách nhau năm mươi mét này thực ra là một chỗ”, và bộ giải — vốn tin mọi ràng buộc — sẽ gập bản đồ lại để thoả mãn nó.

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và nó định hình cả chương: chi phí của một dương tính giả lớn hơn chi phí của một âm tính giả nhiều bậc độ lớn. Bỏ lỡ một loop closure nghĩa là trôi thêm một chút, và sẽ có cơ hội khác ở vòng sau. Chấp nhận một loop closure sai nghĩa là phá huỷ sản phẩm của hàng giờ làm việc, và thường không phục hồi được.

TRỰC GIÁC

Hình dung bạn đi bộ quanh một khu phố, vừa đi vừa vẽ bản đồ bằng cách đếm bước và ghi hướng rẽ.

Sau một vòng, bạn quay về điểm xuất phát — nhưng bản đồ của bạn nói bạn đang ở cách đó ba mươi mét. Sai số đếm bước đã tích lũy.

Bây giờ bạn *nhận ra* cái cửa hàng trước mặt chính là cửa hàng bạn xuất phát. Đây là thông tin cực kỳ quý: nó nói điểm đầu và điểm cuối trùng nhau. Bạn lấy bản đồ ra và *kéo* nó cho hai điểm khớp, phân phối ba mươi mét sai lệch đều ra cả vòng. Bản đồ đột nhiên đúng hơn nhiều — không chỉ ở điểm cuối mà ở *mọi* chỗ.

Đó là loop closure, và nó là phép thuật thật sự của SLAM.

Nhưng bây giờ hình dung bạn nhầm. Cái cửa hàng trước mặt thuộc cùng một chuỗi, cùng biển hiệu, cùng cách bày hàng — nhưng nó ở một khu phố khác cách đó nửa cây số. Bạn kéo bản đồ cho hai điểm khớp.

Kết quả không phải một bản đồ hơi sai. Đó là một bản đồ *gập đôi*: nửa cây số bị nén thành không, hai con phố khác nhau chồng lên nhau, và mọi thứ bạn đã ghi đúng suốt buổi sáng giờ nằm sai chỗ. Và bạn không có cách nào biết, vì bản đồ mới vẫn tự nhất quán — nó nhất quán với một giả thuyết sai.

Chênh lệch giữa hai kịch bản ấy là toàn bộ lý do chương này dài như vậy. Việc nhận ra chỗ cũ thì dễ. Việc *chắc chắn* rằng mình không nhầm mới là bài toán.

1.2 Phát hiện: nhận ra chỗ cũ

A. Vì sao đây là bài toán truy hồi

Điểm khái niệm quan trọng đầu tiên: phát hiện loop closure *không* phải bài toán khớp điểm mà là bài toán *truy hồi* (retrieval). Ta không hỏi “điểm này khớp với điểm nào” mà hỏi “khung này giống khung nào nhất trong hàng chục nghìn khung đã lưu”.

Khác biệt ấy quyết định công cụ. Khớp điểm dùng khoảng cách descriptor và ratio test; truy hồi dùng một *biểu diễn toàn cục* cho mỗi ảnh, một chỉ mục để tìm nhanh, và một thang điểm tương đồng. Đây là bài toán mà cộng đồng tìm kiếm thông tin đã giải cho văn bản, và túi từ trực quan là việc mượn nguyên giải pháp đó.

B. Túi từ trực quan

Quy trình [1], [2]. *Ngoại tuyến*: gom một tập lớn descriptor từ dữ liệu đại diện, phân cụm chúng thành một *từ điển* có cấu trúc cây, mỗi lá là một “từ”. *Trực tuyến*: với mỗi khung, gán mỗi descriptor vào từ gần nhất, thu được một histogram thưa; tính điểm tương đồng giữa hai histogram bằng khoảng cách có trọng số TF-IDF.

Hai chi tiết kỹ thuật đáng biết vì chúng là lý do phương pháp này khả thi.

Chỉ mục ngược (inverse index): với mỗi từ, lưu danh sách các khung chứa nó. Khi truy vấn, chỉ cần duyệt các khung chia sẻ ít nhất một từ với khung hiện tại — thường là một phần rất nhỏ của bản đồ. Đây là thứ đưa chi phí từ tuyến tính theo số khung xuống gần hằng số.

Chỉ mục thuận (direct index): với mỗi khung, lưu descriptor nào thuộc nhánh nào của cây. Khi đã có ứng viên, chỉ so khớp các descriptor thuộc cùng nhánh thay vì so tất cả với tất cả — tăng tốc bước kiểm định hình học ở mục 3 lên nhiều lần. Chi tiết này ít được nhắc nhưng nó là lý do DBoW2 nhanh trong thực tế.

TF-IDF đáng một câu giải thích vì nó là chỗ trực giác đẹp. Một từ xuất hiện trong *mọi* khung — chẳng hạn một mẫu vân sàn phổ biến — không mang thông tin phân biệt nào, nên IDF hạ trọng số của nó gần về không. Một từ hiếm — một vật đặc biệt trong phòng — mang rất nhiều thông

tin. Đây chính là cơ chế mà một người dùng khi tìm đường: bạn không định vị bằng “có tường trắng” mà bằng “có cái cây bonsai kỳ lạ ở góc”.

C. Place recognition học được

NetVLAD [3] và các hệ sau học một biểu diễn toàn cục trực tiếp từ dữ liệu, tối ưu cho việc nhận ra cùng một địa điểm qua các điều kiện khác nhau — mùa, thời tiết, ngày đêm. Đây là cải thiện rõ rệt so với BoW ở điều kiện thay đổi mạnh, vì BoW dựa trên descriptor cục bộ vốn không bền với thay đổi về ngoài lớn.

Tôi *không* dẫn số precision/recall so sánh các hệ ở đây, vì cùng lý do ở B/07 khối *Cam bẫy*: các con số ấy phụ thuộc cực mạnh vào bộ dữ liệu, vào cách định nghĩa “cùng một địa điểm”, và vào ngưỡng. Điều có thể nói trung thực: học được thắng rõ ở *thay đổi về ngoài lớn* (mùa, ngày đêm) và cái giá là GPU cùng tính khó dự đoán khi ra ngoài phân bố huấn luyện.

D. Nhất quán chuỗi

Một kỹ thuật đơn giản và rất hiệu quả, dựa trên một quan sát tầm thường: robot di chuyển liên tục, nên nếu khung t khớp với khung s trong bản đồ, thì khung $t + 1$ nên khớp với khung gần $s + 1$.

SeqSLAM [4] khai thác điều này triệt để: thay vì tìm khớp cho từng ảnh, nó tìm khớp cho cả một *chuỗi* ảnh. Kết quả là nó chịu được những ảnh riêng lẻ rất kém — và nó hoạt động trong điều kiện mà khớp từng ảnh hoàn toàn thất bại.

Ý tưởng chung đáng nhớ hơn thuật toán cụ thể: *ràng buộc thời gian là một nguồn thông tin miễn phí chống lại perceptual aliasing*. Ba mươi cái kệ giống hệt nhau, nhưng chỉ có *một* cái mà robot có thể tới được từ chỗ nó đứng một giây trước. Hầu hết các hệ khai thác điều này ở dạng đơn giản — yêu cầu vài khung liên tiếp cùng đề xuất một vùng — và đó là một trong những phép kiểm rẻ nhất có sẵn.

1.3 Kiểm định hình học: tầng phòng thủ chính

A. Quy trình

Một ứng viên từ truy hồi *chưa phải* một loop closure. Nó phải qua kiểm định hình học, và đây là câu trả lời một phần cho câu hỏi mở đầu thứ hai.

Quy trình: khớp descriptor giữa khung hiện tại và khung ứng viên (dùng chỉ mục thuận để tăng tốc); chạy RANSAC để tìm phép biến đổi — PnP nếu ứng viên có landmark 3D, hoặc $Sim(3)$ nếu hai bên đều có cấu trúc; đếm inlier và đo phần dư.

Vì sao hai trăm inlier vẫn chưa đủ? Ba lý do, và cả ba đều là bài học đã gặp.

Một — perceptual aliasing thật sự. Nếu hai địa điểm giống nhau về mặt hình học *lẫn* về ngoài — hai hành lang xây giống hệt nhau — thì một phép biến đổi nhất quán tồn tại thật, và RANSAC tìm ra nó. Đây là trường hợp mà kiểm định hình học không cứu được, và nó là lý do cần thêm các tầng ở mục B.

Hai — cấu hình suy biến. Như A/01 mục 6 đã nói, số inlier cao tương thích hoàn hảo với một mô hình không xác định. Nếu hai khung đều nhìn vào một bức tường phẳng, phép biến đổi tìm được có thể có một hướng tự do.

Ba — chất lượng phân bố. Hai trăm inlier tụ ở một góc ảnh cho một phép biến đổi có số điều kiện tẻ, đúng như B/07 mục 4A.

B. Bốn tầng phòng thủ

Thiết kế đúng có nhiều tầng, mỗi tầng bắt một loại lỗi khác nhau. Đây là phần còn lại của câu trả lời cho câu hỏi thứ hai.

Tầng một — ngưỡng inlier bảo thủ. Cần thiết nhưng không đủ. Đặt cao hơn nhiều so với ngưỡng dùng cho tracking, vì chi phí lỗi khác nhau.

Tầng hai — nhất quán chuỗi. Yêu cầu nhiều khung liên tiếp cùng đề xuất một vùng bản đồ, với các tư thế tương đối nhất quán với odometry giữa chúng. Rẻ và mạnh — nó tấn công trực tiếp vào perceptual aliasing bằng ràng buộc thời gian.

Tầng ba — nhất quán với hiệp phương sai hiện tại. Nếu hệ thống nghĩ nó đang ở A với bất định ± 5 m, thì một loop closure nói nó đang ở cách A năm mươi mét là *không nhất quán* với chính ước lượng của nó. Về nguyên tắc, đây là phép kiểm mạnh nhất: nó dùng toàn bộ thông tin hệ thống có. Trong thực tế nó bị hạn chế bởi bệnh quá tự tin ở $A/06$ — nếu hiệp phương sai nhỏ hơn sự thật, phép kiểm này sẽ loại cả các loop closure đúng. Cách dùng an toàn là đặt ngưỡng rất rộng, chấp nhận rằng nó chỉ bắt được các trường hợp sai rành rành.

Tầng bốn — nhất quán với các loop closure khác. Nếu một cạnh mâu thuẫn với một nhóm cạnh đã được chấp nhận, nó đáng ngờ. Đây là ý tưởng của RRR [5]: nhóm các loop closure thành các cụm và kiểm tính nhất quán giữa các cụm, loại cả cụm nếu nó mâu thuẫn với phần còn lại. Mạnh nhất trong bốn tầng và cũng đắt nhất.

CẦN NHỚ

Chi phí của hai loại lỗi bất đối xứng tới mức nó phải chi phối mọi quyết định thiết kế trong chương này.

Bỏ lỡ một loop closure: hệ thống trôi thêm một chút, và sẽ có cơ hội khác ở vòng sau. Chi phí tuyến tính và có thể phục hồi.

Chấp nhận một loop closure sai: bản đồ gấp lại, hàng giờ làm việc bị phá huỷ, và thường không phục hồi được vì không có cơ chế nào phát hiện sau đó — bản đồ gấp vẫn tự nhất quán.

Ba hệ quả thiết kế trực tiếp. Một: **ngưỡng phải bảo thủ**, và tỉ lệ dương tính giả nên được đặt làm mục tiêu chứ không phải tỉ lệ phát hiện. Hai: **nhiều tầng phòng thủ độc lập** tốt hơn một tầng tốt, vì các tầng bắt các loại lỗi khác nhau. Ba: **luôn có back-end bền** như tầng cuối, vì không bộ phát hiện nào hoàn hảo.

Và một hệ quả về cách đo: khi đánh giá một hệ loop closure, đường cong precision-recall quan trọng hơn một con số duy nhất, và **vùng precision rất cao** mới là vùng đáng quan tâm. Một hệ có recall 80% ở precision 99% tốt hơn nhiều so với một hệ có recall 95% ở precision 90% — dù con số thứ hai trông đẹp hơn. Chi tiết ở C/18.

1.4 Đóng vòng: áp dụng ràng buộc

A. Vì sao mono cần $Sim(3)$

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và nó là ứng dụng trực tiếp nhất của A/05 mục 3B.

Trong SLAM mono, tỉ lệ là gauge tự do, và nó *trôi*: mỗi lần triangulate một landmark mới từ các landmark cũ, một sai số tỉ lệ nhỏ được đưa vào, và các sai số ấy tích lũy. Sau một vòng dài, tỉ lệ ở cuối vòng có thể lệch vài phần trăm so với ở đầu.

Hệ quả cụ thể nếu dùng $SE(3)$: phép biến đổi $SE(3)$ bảo toàn khoảng cách, nên nó *không thể* ghép hai đầu vòng có tỉ lệ khác nhau. Bộ giải sẽ tìm một thoả hiệp — và thoả hiệp ấy không phải

một sai số nhỏ phân bố đều mà là một *biến dạng* của bản đồ: một phần bị nén, một phần bị kéo, và các ràng buộc odometry bị vi phạm một cách có hệ thống. Đây không phải “kém chính xác hơn” mà là một chế độ hỏng khác về chất.

Với $Sim(3)$ [6], phép biến đổi có bảy bậc tự do gồm cả tỉ lệ, nên nó ghép được hai đầu và *đồng thời* sửa trôi tỉ lệ. Quy trình đầy đủ: tính $Sim(3)$ giữa hai khung, áp dụng nó để hiệu chỉnh các tư thế và landmark trong vùng ảnh hưởng, rồi chạy pose graph $Sim(3)$ để phân phối sai số.

Với stereo hoặc VIO, tỉ lệ quan sát được nên nó không trôi, và $SE(3)$ là đủ — và với VIO thì còn rút xuống bốn bậc vì trọng lực ghim roll với pitch. Số bậc tự do lại một lần nữa là hệ quả của cấu hình cảm biến, không phải một lựa chọn.

B. Pose graph và phân phối sai số

Sau khi có ràng buộc, chạy tối ưu pose graph (B/09 mục 4). Điều quan trọng cần hiểu: tối ưu này *không* làm cho sai số biến mất — nó *phân phối lại* sai số một cách tối ưu theo trọng số.

Trực giác: ba mươi mét sai lệch ở điểm cuối không phải lỗi của riêng bước cuối. Nó là tổng của hàng nghìn sai số nhỏ dọc đường. Pose graph phân bổ phần sửa cho từng cạnh theo tỉ lệ nghịch với độ tin cậy của nó — cạnh có hiệp phương sai lớn nhận phần sửa nhiều hơn. Đây chính là bài toán MAP đang làm đúng việc của nó, và nó cho thấy vì sao hiệp phương sai của các cạnh odometry phải đúng: nếu chúng sai, phần sửa được phân bổ sai chỗ.

C. Hoà nhập với luồng tracking

Một vấn đề kỹ thuật thật mà ít tài liệu học thuật nói tới, và nó thuộc về C/13 nhiều hơn nhưng cần nêu ở đây.

Tối ưu pose graph mất từ hàng chục mili giây tới hàng giây tùy kích thước; BA toàn cục sau đó còn lâu hơn. Trong lúc đó, luồng tracking *vẫn phải chạy* — không được dừng vì camera vẫn gửi khung và robot vẫn di chuyển. Nhưng tracking đang dùng bản đồ mà loop closing sắp sửa đổi.

Ba cách xử lý trong thực tế. *Đóng băng*: dừng tracking trong lúc cập nhật — đơn giản, và không chấp nhận được nếu vòng điều khiển phụ thuộc vào tư thế. *Sao chép*: loop closing làm việc trên một bản sao, rồi hoán đổi nhanh; tốn bộ nhớ và cần xử lý các khung xử lý trong lúc đó. *Cập nhật gia tăng*: áp dụng phần hiệu chỉnh dần dần. ORB-SLAM3 dùng một biến thể của cách thứ hai với một giai đoạn hoà nhập cẩn thận [7].

Điều đáng nhớ: đây là loại vấn đề *không xuất hiện trong bài báo nhưng chiếm phần lớn thời gian kỹ thuật*. Nó là một trong những lý do khoảng cách giữa một bài báo và một sản phẩm lớn hơn về ngoài, và là chủ đề của D/20.

1.5 Back-end bền: sống sót khi đã nhầm

Không bộ phát hiện nào hoàn hảo. Tầng cuối là làm cho bài toán tối ưu chịu đựng được các cạnh sai.

A. Ba họ kỹ thuật

Switchable constraints [8]: gán cho mỗi cạnh loop closure một biến “công tắc” $s \in [0, 1]$ và đưa nó vào bài toán tối ưu, với một prior kéo s về một. Nếu một cạnh mâu thuẫn mạnh với phần còn lại, bộ giải thấy rẻ hơn khi hạ s xuống — tức tự tắt cạnh đó. Ý tưởng đẹp: để bộ giải tự quyết định tin cạnh nào, thay vì quyết định trước.

DCS [9]: cùng ý tưởng nhưng loại bỏ biến công tắc bằng cách giải cho nó ở dạng đóng và thay vào — kết quả là một hàm mất mát bền có dạng cụ thể. Rẻ hơn và dễ cài hơn, cho hành vi tương tự.

GNC [10]: bắt đầu với hàm mất mát gần lỗi rồi làm nó không lỗi dần (A/04 mục 6C). Giải bài toán dễ trước để có khởi tạo tốt, rồi dùng nó cho bài toán khó hơn. Mạnh hơn hai cách trên khi tỉ lệ outlier cao.

Cả ba là các biến thể của cùng một ý tưởng, và đối ngẫu Black–Rangarajan [11] là chỗ sự thống nhất ấy được phát biểu: tối ưu với một hàm mất mát bền *tương đương* với tối ưu đồng thời trên biến gốc và một tập biến trọng số outlier. Switchable constraints làm điều đó một cách tường minh; DCS và M-estimator làm ngầm.

B. Điều kiện để nó hoạt động

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó là giới hạn quan trọng nhất của mục này.

Back-end bền hoạt động khi các cạnh sai là *thiếu số* và *không nhất quán với nhau*. Lúc đó, đa số đúng tạo ra một “sự thật” mà thiếu số sai mâu thuẫn với, và bộ giải tắt thiếu số.

Điều kiện ấy bị vi phạm trong đúng tình huống khó nhất: *perceptual aliasing có hệ thống*. Trong hành lang kho hàng với ba mươi kệ giống nhau, các loop closure sai không ngẫu nhiên — chúng đều nói “bạn đang ở kệ nào đó”, và chúng *nhất quán với nhau* theo một cách sai. Nếu số cạnh sai vượt số cạnh đúng, back-end bền sẽ tắt các cạnh *đúng*. Nó không chỉ thất bại mà thất bại theo hướng ngược lại.

Đây là lý do back-end bền là tầng *cuối cùng* chứ không phải tầng duy nhất, và là lý do bốn tầng ở mục 3B không thể bỏ qua. Nó cũng là lý do vì sao perceptual aliasing được liệt kê ở D/22 như một bài toán chưa giải quyết: nó phá vỡ giả định nền của mọi kỹ thuật bền.

C. Hai nguồn thông tin ngoài về ngoài

Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Trong môi trường lặp lại có hệ thống, hai nguồn vẫn phân biệt được.

Một — ràng buộc thời gian và odometry. Robot không dịch chuyển tức thời. Nếu một giây trước nó ở kệ thứ năm, nó không thể đang ở kệ thứ hai mươi. Đây là thông tin mạnh và miễn phí, và nó là cơ sở của tầng hai và tầng ba ở mục 3B. Giới hạn: nó chỉ giúp khi odometry còn đáng tin — tức đúng lúc ta chưa cần loop closure. Sau tám tiếng trôi, bất định đã lớn tới mức ràng buộc này không loại được gì.

Hai — neo tuyệt đối. Một marker fiducial đặt ở các vị trí đã biết, một tag UWB, một tín hiệu wifi. Đây là cách mà phần lớn hệ thống kho hàng thật giải quyết vấn đề, và nó là một lời thú nhận đáng chú ý: *bài toán thuần thị giác trong môi trường lặp lại chưa có lời giải tốt, nên công nghiệp giải nó bằng cách thay đổi môi trường*. Cây `./marker/` bàn về nhánh này. Giới hạn: nó đòi sửa đổi cơ sở hạ tầng, không phải lúc nào cũng làm được, và nó chuyển bài toán sang việc bảo trì các neo đó.

Cả hai đều không phải lời giải hoàn chỉnh, và đó là phát biểu trung thực nhất tôi đưa ra được.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [7]	DBoW2 + kiểm định $Sim(3)$ + pose graph + BA toàn cục; Atlas ghép nhiều bản đồ	Cài đặt đầy đủ nhất; đọc phần hoà nhập với tracking ở mục 4C
VINS-Fusion [12]	Pose graph 4-DoF riêng biệt với luồng VIO	Kiến trúc hai tầng; 4-DoF vì có IMU, đúng A/05 mục 3B

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DBoW2 [1]	Từ điển nhị phân, chỉ mục ngược và thuận	Đọc để hiểu chỉ mục thuận — chỉ tiết ít được nhắc nhưng quyết định tốc độ
RRR [5]	Phân cụm loop closure và kiểm nhất quán giữa các cụm	Tầng bốn ở mục 3B — mạnh nhất và đắt nhất
GTSAM [10]	GNC cài sẵn cho pose graph	Mục 5A; dễ thử nhất trong ba họ
maplab [13]	Loop closure đa phiên; quản lý bản đồ dài hạn	Hiếm framework xử lý tốt mảng nhiều phiên — xem C/13

MINI PROJECT SLAM · Đo đường cong precision-recall của hệ loop closure, rồi cố tình tìm cạnh sai · 4 ngày

Mục tiêu	chuyển từ “hệ của tôi đóng vòng được” sang “hệ của tôi có tỉ lệ dương tính giả là X ở ngưỡng Y , và nó sống sót được $Z\%$ cạnh sai”.
Dữ liệu	một chuỗi có loop thật và có chân trị (KITTI hoặc EuRoC); cộng một chuỗi tự quay trong môi trường lặp lại — một hành lang, một dãy phòng giống nhau. Chuỗi thứ hai là phần quan trọng.
Các bước	<i>Phần một:</i> cài phát hiện bằng BoW, ghi lại điểm số cho mọi cặp khung, dùng chân trị để gán nhãn đúng/sai, vẽ đường cong precision-recall; chú ý vùng precision > 99%. <i>Phần hai:</i> thêm từng tầng phòng thủ ở mục 3B và vẽ lại đường cong sau mỗi tầng. <i>Phần ba:</i> chạy pose graph với các cạnh đã tìm thêm $k\%$ cạnh sai ngẫu nhiên, với k từ 0 tới 50; so ATE giữa pose graph thường, với Huber, và với GNC.
Xong khi	bạn có ba kết quả. (1) Một đường cong precision-recall và một ngưỡng chọn từ vùng precision cao chứ không từ điểm F1 tối đa — và bạn giải thích được vì sao. (2) Một bảng cho thấy mỗi tầng phòng thủ đóng góp bao nhiêu, và tầng nào rẻ nhất trên mỗi đơn vị cải thiện. (3) Một đường cong ATE theo tỉ lệ cạnh sai, cho thấy GNC giữ được tới một ngưỡng rồi sụp — và bạn xác định được ngưỡng đó, vốn chính là điều kiện ở mục 5B.
Nối với	C/13-quan-ly-ban-do xử lý vấn đề hoà nhập ở mục 4C; C/18-danh-gia-va-phuong-phap-luan chuẩn hoá cách đo precision-recall; D/22-huong-nghien-cuu-con-trong lấy perceptual aliasing làm một trong tám khoảng trống.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Tỉ số ít nhất vài bậc độ lớn, và nó bất đối xứng về *bản chất* chứ không chỉ về độ lớn. Bỏ lỡ: trôi thêm một chút, có cơ hội khác ở vòng sau, phục hồi được. Chấp nhận sai: bản đồ gấp lại, hàng giờ làm việc bị phá, và *không phục hồi được* vì bản đồ gấp vẫn tự nhất quán nên không cơ chế nào phát hiện sau đó. Ba hệ quả thiết kế: ngưỡng phải bảo thủ và mục tiêu nên là tỉ lệ dương tính giả chứ không phải tỉ lệ phát hiện; nhiều tầng phòng thủ độc lập tốt hơn một tầng tốt; luôn phải có back-end bền lâu làm tầng cuối. Và khi đó, vùng precision rất cao mới là vùng đáng quan tâm, không phải điểm F1 tối đa.
- A2. Vì ba lý do. (a) *Perceptual aliasing thật sự*: hai hành lang xây giống hệt nhau thì một phép biến đổi nhất quán tồn tại thật, và RANSAC tìm ra nó — kiểm định hình học không cứu được. (b) *Cấu hình suy biến*: nếu cả hai khung nhìn vào tường phẳng, phép biến đổi có thể có hướng tự do, và số inlier cao tương thích hoàn hảo với điều đó (A/01 mục 6). (c) *Phân bố kém*: hai trăm inlier tự một góc cho phép biến đổi có số điều kiện tẻ. Phép kiểm bổ sung, theo thứ tự rẻ: nhất quán chuỗi (nhiều khung liên tiếp cùng đề xuất một vùng, với tư thế tương đối khớp odometry), nhất quán với hiệp phương sai hiện tại, và nhất quán với các loop closure đã chấp nhận (RRR).
- A3. Hai nguồn: (a) *ràng buộc thời gian và odometry* — robot không dịch chuyển tức thời, nên chỉ một trong ba mươi kệ là tới được từ chỗ nó đứng một giây trước; (b) *neo tuyệt đối* — marker, UWB, wifi. Cả hai đều không hoàn chỉnh. Nguồn (a) chỉ giúp khi odometry còn đáng tin, tức đúng lúc ta chưa cần loop closure; sau tám tiếng trôi, bất định đã lớn tới mức nó không loại được gì. Nguồn (b) đòi sửa đổi cơ sở hạ tầng và chuyển bài toán sang việc bảo trì các neo. Việc công

nghiệp chủ yếu dùng (b) là một lời thú nhận đáng chú ý: bài toán thuần thị giác trong môi trường lặp lại chưa có lời giải tốt.

- A4. Cụ thể là bản đồ bị *biến dạng*, không phải “kém chính xác hơn”. Trong mono, tỉ lệ trôi, nên hai đầu vòng có tỉ lệ khác nhau vài phần trăm. Phép biến đổi $SE(3)$ bảo toàn khoảng cách nên nó không ghép được hai đầu có tỉ lệ khác nhau; bộ giải tìm một thoả hiệp, và thoả hiệp ấy nén một phần bản đồ và kéo phần khác, vi phạm các ràng buộc odometry một cách có hệ thống. $Sim(3)$ có bảy bậc gồm cả tỉ lệ nên nó ghép được và *đồng thời* sửa trôi tỉ lệ [6]. Với stereo hoặc VIO thì tỉ lệ quan sát được nên không trôi, và $SE(3)$ — hoặc bốn bậc với VIO — là đủ.
- A5. Điều kiện: các cạnh sai phải là *thiếu số* và *không nhất quán với nhau*. Lúc đó đa số đúng tạo ra một “sự thật” mà thiếu số sai mâu thuẫn với, và bộ giải tắt thiếu số. Điều kiện bị vi phạm trong đúng tình huống khó nhất: *perceptual aliasing có hệ thống*. Trong hành lang ba mươi kệ giống nhau, các cạnh sai không ngẫu nhiên — chúng nhất quán với nhau theo một cách sai. Nếu chúng vượt số cạnh đúng, back-end bền sẽ tắt các cạnh *đúng*, tức thất bại theo hướng ngược lại. Đó là lý do nó là tầng cuối chứ không phải tầng duy nhất, và là lý do perceptual aliasing nằm trong danh sách bài toán chưa giải ở D/22.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đi bộ quanh khu phố và vẽ bản đồ bằng cách đếm bước. Quay về chỗ xuất phát, bản đồ nói bạn còn cách đó ba mươi mét — vì đếm bước có sai số và sai số cộng dồn.

Nhưng bạn *nhận ra* cửa hàng trước mặt chính là cửa hàng lúc xuất phát. Thế là bạn lấy bản đồ và kéo nó cho hai điểm trùng nhau, chia đều ba mươi mét sai lệch ra cả vòng. Bản đồ đột nhiên đúng hơn hẳn — không chỉ ở điểm cuối mà ở mọi chỗ. Đây là phép thuật thật sự, và nó là lý do chữ M trong SLAM tồn tại.

Bây giờ hình dung bạn nhầm. Cửa hàng trước mặt cùng chuỗi, cùng biển hiệu, nhưng ở khu phố khác cách đó nửa cây số. Bạn vẫn kéo bản đồ cho hai điểm trùng.

Kết quả không phải một bản đồ hơi sai. Đó là một bản đồ *gập đôi*: nửa cây số bị nén thành không, hai con phố chồng lên nhau, và mọi thứ bạn ghi đúng suốt buổi sáng giờ nằm sai chỗ. Tệ nhất: bạn không có cách nào biết, vì bản đồ mới *vẫn tự nhất quán* — chỉ là nó nhất quán với một giả thuyết sai.

Đó là lý do chương này dài. Nhận ra chỗ cũ thì dễ. Chắc chắn rằng mình không nhầm mới là bài toán.

Và cách chắc chắn không phải nhìn kỹ hơn vào cửa hàng. Đó là hỏi thêm vài câu khác: *Tôi có thể đi từ chỗ tôi đứng một phút trước tới đây trong một phút không? Nếu đây thật là cửa hàng xuất phát, thì mọi thứ khác trong bản đồ có còn hợp lý không? Mấy phút vừa qua tôi có liên tục thấy những thứ quen thuộc, hay chỉ đúng một khoảnh khắc này?*

Còn một chỗ mà mọi câu hỏi đều thất bại: một dãy kho hàng có ba mươi cửa giống hệt nhau, cách nhau ba mét. Ở đó bạn không phân biệt được, và cũng không có gì trong khung cảnh giúp bạn phân biệt. Đó là bài toán khó nhất trong lĩnh vực này, và câu trả lời thật thà là nó chưa được giải.

Chỗ không rút gọn được: chỗ không rút gọn được là chính cái dãy kho hàng ấy. Mọi kỹ thuật trong chương này đều giả định rằng “số đồng thì đúng”. Nhưng khi môi trường lặp lại một cách có hệ thống, số đồng cũng có thể sai một cách nhất quán, và toàn bộ lập luận sụp đổ. Cách duy nhất thoát ra là thêm thông tin từ bên ngoài thị giác — và đó là lý do các nhà kho thật dán marker lên tường.

Một câu: *Đóng vòng đúng sửa cả một ngày làm việc; đóng vòng sai phá cả một ngày làm việc — và hai việc đó trông giống hệt nhau ở thời điểm quyết định.*

1.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *chi phí của dương tính giả lớn hơn âm tính giả nhiều bậc* — là một lập luận cấu trúc, và nó bị bác bỏ nếu một hệ có cơ chế *phát hiện và huỷ* một loop closure sai sau khi đã áp dụng. Điều đó không bất khả: RRR [5] làm gần như vậy bằng cách kiểm nhất quán giữa các cụm

và có thể loại cả cụm đã chấp nhận. Nếu một cơ chế như vậy đủ tin cậy, sự bất đối xứng giảm và ngưỡng có thể nới. Tôi cho rằng nó chưa đủ tin cậy trong thực tế, nhưng đây là chỗ tôi có ít bằng chứng nhất (*tôi suy ra, chưa đối chiếu nguồn*).

Mệnh đề thứ hai — *mono cần $Sim(3)$, stereo cần $SE(3)$, VIO cần bốn bậc* — kế thừa từ A/05 mục 3B và kiểm được bằng cách chạy pose graph với số bậc sai rồi xem biến dạng. Nó bị bác bỏ nếu một hệ mono dùng $SE(3)$ mà không biểu hiện biến dạng — điều sẽ xảy ra nếu trôi tỉ lệ đủ nhỏ, chẳng hạn vòng rất ngắn hoặc có prior tỉ lệ ngầm từ đâu đó.

Mệnh đề thứ ba — *back-end bền cần cạnh sai là thiếu số và không nhất quán* — là điều kiện được nêu trong các công trình gốc. Điều tôi *chưa* kiểm là ngưỡng định lượng: bao nhiêu phần trăm cạnh sai thì GNC sụp? Mini project phần ba đo đúng điều đó, và con số thu được sẽ phụ thuộc mạnh vào cấu trúc đồ thị. Đây là một mục trong 99-chua-biet.md.

Một mệnh đề mềm và là mệnh đề tôi ít chắc nhất: chương này khẳng định *perceptual aliasing có hệ thống chưa có lời giải tốt*. Nó bị bác bỏ nếu một hệ đạt precision cao trong môi trường lặp lại có hệ thống mà chỉ dùng thị giác, được kiểm định trên dữ liệu thật chứ không phải benchmark. Các mô hình học được gần đây khai thác ngữ cảnh rộng hơn, nên ranh giới đang dịch chuyển, và tôi không chắc phát biểu này còn đúng trong hai năm nữa.

1.7 Kết nối

Nổi lên trên. B/07-dac-trung-va-data-association là chương mà bài toán này xuất hiện lần đầu ở quy mô nhỏ; điều đáng chú ý là *cùng một nguyên lý* — thông tin để phát hiện khớp sai nằm trong quan hệ với các khớp khác — áp dụng ở cả hai quy mô, chỉ khác hậu quả. B/09-paradigm-uoc-luong-back-end cho pose graph và giải thích vì sao MSCKF không đóng vòng được. A/03-lie-group-va-toi-uu-tren-da-tap cho $Sim(3)$; A/04-uoc-luong-map-va-bundle-adjustment cho GNC và M-estimator; A/05-observability-gauge-va-suy-bien cho số bậc tự do và cho lý do trôi tỉ lệ tồn tại.

Nổi xuống dưới. B/12-relocalization dùng cùng công nghệ phát hiện — BoW, kiểm định hình học — cho một mục đích khác, và có cùng cấu trúc chi phí bất đối xứng. C/13-quan-ly-ban-do xử lý vấn đề hoà nhập ở mục 4C và mở rộng sang ghép nhiều bản đồ, nhiều phiên. C/18-danh-gia-va-phuong-phap-luan chuẩn hoá cách đo precision-recall và lập luận vì sao vùng precision cao mới quan trọng.

Nổi xa hơn. D/19-ben-vung-trong-the-gioi-thuc bàn về loop closure khi môi trường đã thay đổi — cùng một chỗ nhưng đồ đạc đã dịch, ánh sáng đã khác. D/22-huong-nghien-cuu-con-trong lấy perceptual aliasing làm một trong tám khoảng trống, và mục 5B của chương này là lập luận vì sao nó khó: nó phá vỡ giả định nền của mọi kỹ thuật bền.

Nổi ngang: ../marker/ là nhánh giải bài toán này bằng cách thay đổi môi trường — đặt các neo có danh tính duy nhất. Đây là lời giải thực dụng nhất hiện có cho môi trường lặp lại, và đọc cây đó cho thấy cái giá của nó: hiệu chuẩn, bảo trì, và giới hạn tầm nhìn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ước lượng tỉ số chi phí giữa loop closure sai và loop closure bỏ lỡ, rồi nêu ba hệ quả thiết kế của nó.
2. Vì sao phát hiện loop closure là bài toán truy hồi chứ không phải bài toán khớp, và điều đó quyết định công cụ thể nào?
3. Nêu bốn tầng phòng thủ chống loop closure sai, và nói mỗi tầng bắt được loại lỗi nào mà ba tầng kia bỏ sót.

4. Dùng $SE(3)$ thay cho $Sim(3)$ trong SLAM mono. Mô tả *cụ thể* bản đồ sẽ trông thế nào, không dùng cụm từ “kém chính xác hơn”.
5. Pose graph không làm sai số biến mất mà phân phối lại nó. Theo nguyên tắc nào, và điều gì phải đúng thì việc phân phối mới đúng?
6. Nêu điều kiện để back-end bền hoạt động, và mô tả một tình huống thực tế mà điều kiện đó bị vi phạm.
7. Khi đánh giá hệ loop closure, vì sao điểm F1 tối đa là tiêu chí sai, và tiêu chí nào đúng?

Nguồn

- [1] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012, mức tin A.
- [2] D. Nistér and H. Stewénus, “Scalable Recognition with a Vocabulary Tree,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2006.
- [3] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1437–1451, 2018, mức tin A.
- [4] M. J. Milford and G. F. Wyeth, “SeqSLAM: Visual Route-Based Navigation for Sunny Summer Days and Stormy Winter Nights,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2012.
- [5] Y. Latif, C. Cadena, and J. Neira, “Robust Loop Closing over Time for Pose Graph SLAM,” *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013, mức tin A; thuật toán RRR.
- [6] H. Strasdat, J. M. M. Montiel, and A. J. Davison, “Scale Drift-Aware Large Scale Monocular SLAM,” in *Robotics: Science and Systems (RSS)*, mức tin A; nguồn của loop closure trên $Sim(3)$, 2010.
- [7] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [8] N. Sunderhauf and P. Protzel, “Switchable Constraints for Robust Pose Graph SLAM,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2012.
- [9] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., “Robust Map Optimization Using Dynamic Covariance Scaling,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2013.
- [10] H. Yang, P. Antonante, V. Tzoumas, et al., “Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020, mức tin A.
- [11] M. J. Black and A. Rangarajan, “On the Unification of Line Processes, Outlier Rejection, and Robust Statistics with Applications in Early Vision,” *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–91, 1996, mức tin A; nguồn của đối ngẫu Black-Rangarajan.
- [12] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [13] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.